AI Agent 技术白皮书

企业智能化转型的核心技术框架

版本 2.0 · 2026年4月

LangGraph-Claude 技术团队 编著

第一章 AI Agent 概述

1.1 什么是 AI Agent

AI Agent（人工智能代理）是一种能够感知环境、制定决策并采取行动以实现特定目标的智能系统。与传统的问答系统不同，AI Agent 具备自主推理、工具调用和多步骤规划能力。它可以将复杂任务分解为子任务，循环执行直至达成目标。

AI Agent 的三大核心能力：

- 感知（Perception）：通过多种输入源获取环境信息，包括文本、图像、结构化数据等。
- 推理（Reasoning）：基于大语言模型进行逻辑推理，分解问题，制定行动计划。
- 行动（Action）：调用外部工具（搜索、计算、API）执行具体操作并获取反馈。

1.2 Agent 与普通 LLM 应用的区别

普通 LLM 应用采用单次问答模式（输入→输出），而 Agent 采用循环推理模式（思考→行动→观察→再思考）。这种循环使得 Agent 能够处理需要多步骤推理的复杂任务，如：调研报告生成、代码调试、数据分析等。Agent 的行动空间由其配置的工具集决定，可以无限扩展。

1.3 Agent 的主要架构类型

目前业界主流的 Agent 架构类型分为以下四种：

- ReAct（Reasoning + Acting）：交替进行推理和行动，是最经典的单 Agent 模式，适合工具调用场景。
- Plan-and-Execute：先制定完整计划，再逐步执行，适合长任务规划场景。
- Multi-Agent：多个专业化 Agent 协作，通过消息传递和状态共享完成复杂任务。
- Human-in-the-Loop：在关键决策节点引入人工审核，适合高风险业务场景。

第二章 RAG 技术详解

2.1 RAG 的定义与原理

RAG（Retrieval-Augmented Generation，检索增强生成）是一种将信息检索与文本生成相结合的技术框架。其核心思想是：在 LLM 生成回答之前，先从外部知识库中检索相关文档，将检索结果作为上下文注入 Prompt，从而使 LLM 能够基于最新、准确的私有知识回答问题，有效解决 LLM 的幻觉问题和知识截止问题。

2.2 RAG 完整流程

RAG 系统分为两个主要阶段：索引阶段（Indexing）和检索生成阶段（Retrieval & Generation）。

索引阶段步骤：

① 文档加载（Document Loading）：支持 PDF、Word、网页、数据库等多种数据源。

② 文本分块（Text Splitting）：将长文档切割为合适大小的文本块（Chunk），通常 200-1000 字符。

③ 向量化（Embedding）：使用 Embedding 模型将文本块转化为高维向量表示。

④ 存储（Storage）：将向量及元数据存入向量数据库（如 Chroma、Pinecone、Milvus）。

检索生成阶段步骤：

① 问题向量化：将用户问题转化为向量。

② 相似度检索：在向量库中找出最相关的 Top-K 文档块。

③ 上下文构建：将检索结果拼接为 Context，注入 Prompt。

④ LLM 生成：基于 Context 生成准确答案。

2.3 混合检索策略

单纯的语义检索（Dense Retrieval）在某些场景表现不佳，例如精确的关键词匹配、专业术语查询等。生产环境推荐采用混合检索策略：

语义检索（Dense）+ 关键词检索（BM25 Sparse）结合，通过 RRF（Reciprocal Rank Fusion）算法融合两种检索结果。语义检索捕捉语义相关性，BM25 保证关键词精确匹配，两者互补，显著提升召回率和准确率。根据实验数据，混合检索相比单一语义检索，准确率平均提升 15-20%。

2.4 高级 RAG 技术

当前业界常用的高级 RAG 优化技术包括：

- Query Rewriting（问题改写）：多轮对话中，将含代词的问题改写为独立完整问题，提升检索准确性。

- HyDE（假设文档嵌入）：让 LLM
生成一个假设答案，用该答案的向量检索，而非用问题向量检索。
- Context Compression（上下文压缩）：从检索到的文档块中只提取与问题直接相关的句子，减少噪声。
- Reranker（重排序）：用交叉编码器对召回结果重新打分排序，精度优于双塔模型。
- Self-RAG：让模型自主决定何时检索、检索什么、如何使用检索结果。

第三章 技术选型与最佳实践

3.1 LLM 选型建议

截至 2026 年，主流 LLM 提供商及其适用场景：

模型	提供商	特点	适用场景
claude-sonnet-4-6	Anthropic	推理强、长上下文、中文优秀	生产主力，复杂推理
claude-haiku-4-5	Anthropic	速度快、成本低	简单任务、大量调用
gpt-4o	OpenAI	多模态、生态丰富	代码生成、多模态
gpt-4o-mini	OpenAI	低成本	简单分类、摘要
Qwen2.5-72B	阿里云	中文最强开源	国内部署、私有化
DeepSeek-V3	DeepSeek	高性价比	成本敏感场景

3.2 向量数据库选型

根据使用场景选择合适的向量数据库：

- Chroma：纯 Python，零配置，适合本地开发和中小规模（<100万向量）。
- Pinecone：全托管云服务，适合快速上线、无运维负担场景。
- Milvus：高性能分布式，适合大规模（>1亿向量）生产场景。
- pgvector：PostgreSQL 扩展，适合已有 PG 基础设施的团队，减少技术栈复杂度。

3.3 Embedding 模型选型

Embedding 质量直接影响检索效果。主流 Embedding 模型对比：

- text-embedding-3-small（OpenAI）：1536维，性价比高，中英文均衡，推荐首选。
- text-embedding-3-large（OpenAI）：3072维，精度更高，成本约3倍。
- BGE-M3（智源研究院）：多语言、多粒度，中文表现优秀，支持本地部署。
- Cohere Embed v3：多语言支持强，适合国际化场景。

第四章 产品规格与性能基准

4.1 RAG 系统性能指标

以下为企业级 RAG 系统的生产环境性能基准（基于 claude-sonnet-4-6 + Chroma + 混合检索）：

指标	目标值	说明
单次问答延迟	≤ 5 秒	P90 延迟，含检索+生成
文档索引速度	≤ 30 秒/100页	PDF解析+分块+向量化
检索准确率（MRR@4）	≥ 0.75	Top-4 召回中包含正确答案
答案忠实度（Faithfulness）	≥ 0.85	回答基于检索内容，无幻觉
并发支持	50+ 用户	同时发起问答请求
知识库容量	10万+ Chunks	单实例 Chroma 支持

4.2 产品定价与套餐

RAG 知识库产品采用 SaaS 订阅模式，提供三个套餐：

- 基础版（¥299/月）：5个知识库，50MB存储，每月50万 Token，支持3用户。
- 专业版（¥999/月）：20个知识库，500MB存储，每月300万 Token，支持20用户，API访问。
- 企业版（¥4999/月）：无限知识库，10GB存储，每月2000万 Token，无限用户，私有部署选项。

4.3 安全与合规

企业级 RAG 系统需满足以下安全合规要求：

- 数据隔离：多租户场景下，各用户知识库完全隔离，向量索引按租户分区。
- 传输加密：所有 API 通信使用 TLS 1.3 加密。
- 访问控制：基于 RBAC 的权限管理，文档级别的访问权限控制。
- 审计日志：记录所有查询请求、文档操作，日志保留90天。
- GDPR 合规：支持用户数据删除请求，数据在中国大陆服务器存储（信创合规）。

第五章 实施路线图与最佳实践

5.1 企业 RAG 落地路线图

Phase 1 — 概念验证（1-2周）：

选取 10-20 份代表性文档，搭建最小可用 RAG 系统，验证技术可行性和效果。

Phase 2 — 试点上线（2-4周）：

接入完整知识库，完善问答质量评估，对接企业 SSO，在特定团队试点。

Phase 3 — 规模推广（1-3月）：

全员推广，建立知识库运营体系，持续优化 Prompt 和检索策略，建立反馈闭环。

5.2 常见问题与解决方案

问题1：回答不准确，引用了错误内容（幻觉问题）

解决：降低生成温度（temperature=0），强化 System Prompt 的约束，增加 Faithfulness 评估。

问题2：检索召回率低，相关文档未被找到

解决：引入混合检索（语义+BM25），调整 chunk_size，尝试 HyDE 或问题扩展技术。

问题3：多轮对话中代词理解错误

解决：在检索前增加 Query Rewriting 步骤，将代词还原为具体实体再检索。

问题4：知识库更新后旧答案仍被召回

解决：文档更新时重新生成对应 chunk 的向量（增量更新），避免全量重建。

5.3 评估体系

生产 RAG 系统必须建立完整的评估体系，核心指标包括：

- Faithfulness（忠实度）：回答是否完全基于检索内容，无幻觉。
- Answer Relevancy（答案相关性）：回答是否切题，有无偏题。
- Context Precision（上下文精确率）：检索到的内容是否都有用。
- Context Recall（上下文召回率）：相关内容是否都被检索到。

推荐使用 RAGAS 框架自动化评估，结合 LangSmith 追踪每次问答的完整链路。

© 2026 LangGraph-Claude 技术团队 · 本文档仅供内部学习使用